

Week 8: Database Relationship Analysis Using Joins

-Abinaya.R (E25AI102)

SCENARIO

The medical shop maintains information about medicines, suppliers, customers, sales, and stock details in a database. The management needs an efficient system to store and retrieve information, monitor medicine availability, track sales transactions, and identify medicines that need to be restocked. SQL queries are used to manage the database, retrieve required records, analyze sales details, and support the smooth and effective daily operations of the medical shop.

Implement INNER JOIN

INNER JOIN returns only matching records from both tables.

1.Display customer details along with their orders.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT, O.ORDER_STATUS FROM CUSTOMER C INNER JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID;
```

CUSTO	CUSTOMER_NAME	ORDER_I	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
C001	Devi	0101	01-SEP-26	1500	Pending
C002	Radhi	0102	02-SEP-26	2500	Shipped
C003	Ajay	0103	03-SEP-26	1800	Delivered

CUSTO	CUSTOMER_NAME	ORDER_I	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
C004	Vijay	0104	04-SEP-26	3200	Pending
C005	Abi	0105	05-SEP-26	2750	Shipped
C006	Anu	0106	06-SEP-26	1200	Delivered

CUSTO	CUSTOMER_NAME	ORDER_I	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
C007	Sindhu	0107	07-SEP-26	4500	Pending
C009	Sanjay	0108	08-SEP-26	2100	Shipped

2. Display order details with payment information.

```
SQL> SELECT O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT, O.ORDER_STATUS, P.PAYMENT_ID, P.PAYMENT_DATE, P.PAYMENT_MODE, P.PAYMENT_STATUS, P.TRANSACTION_AMOUNT FROM ORDERS O INNER JOIN PAYMENT P ON O.ORDER_ID = P.ORDER_ID;
```

ORDER_I	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS	PAYMENT_ID	PAYMENT_D
0101	01-SEP-26	1500	Pending	1	20-SEP-26
0102	02-SEP-26	2500	Shipped	2	21-SEP-26
0103	03-SEP-26	1800	Delivered	3	22-SEP-26

ORDER_I	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS	PAYMENT_ID	PAYMENT_D
0104	04-SEP-26	3200	Pending	4	23-SEP-26
0105	05-SEP-26	2750	Shipped	5	24-SEP-26

3. Retrieve products purchased by customers.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, O.ORDER_ID, P.PRODUCT_ID, P.PRODUCT_NAME, OD.QUANTITY, OD.PRICE FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID JOIN ORDER_DETAILS OD ON O.ORDER_ID = OD.ORDER_ID JOIN PRODUCT P ON OD.PRODUCT_ID = P.PRODUCT_ID;
```

CUSTO	CUSTOMER_NAME	ORDER_I	PRODUCT	PRODUCT_NAME	QUANTITY	PRICE
C001	Devi	0101	P101	Paracetamol	2	750
C002	Radhi	0102	P102	Amoxicillin	3	833.33
C003	Ajay	0103	P103	Cetirizine	2	900
C004	Vijay	0104	P104	Omeprazole		

Implement LEFT JOIN

1. Display all customers including customers who have not placed orders.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID;
```

CUSTO	CUSTOMER_NAME	ORDER_I	ORDER_DAT	TOTAL_AMOUNT
C001	Devi	0101	01-SEP-26	1500
C002	Radhi	0102	02-SEP-26	2500
C003	Ajay	0103	03-SEP-26	1800
C004	Vijay	0104	04-SEP-26	3200
C005	Abi	0105	05-SEP-26	2750
C006	Anu	0106	06-SEP-26	1200
C007	Sindhu	0107	07-SEP-26	4500
C008	Sanjay	0108	08-SEP-26	2100
C001	Devi	0109	09-SEP-26	3500
C002	Radhi	0110	10-SEP-26	1900

2. Find customers without any purchases.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID WHERE O.ORDER_ID IS NULL;
```

no rows selected

3. Display all products including products with no sales.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, OD.ORDER_ID, OD.QUANTITY FROM PRODUCT P LEFT JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID;
```

PRODUCT	PRODUCT_NAME	ORDER_I	QUANTITY
P101	Paracetamol	0101	2
P102	Amoxicillin	0102	3
P103	Cetirizine	0103	2
P104	Omeprazole		

Implement RIGHT JOIN

1. Display all orders with customer information about their product.

```
SQL> SELECT O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT, O.ORDER_STATUS, C.CUSTOMER_ID, C.CUSTOMER_NAME FROM CUSTOMER C RIGHT JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID;
```

ORDER_I	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS	CUSTO
CUSTOMER_NAME				

0101	01-SEP-26	1500	Pending	C001
Devi				
0102	02-SEP-26	2500	Shipped	C002
Radhi				
0103	03-SEP-26	1800	Delivered	C003
Ajay				

ORDER_I	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS	CUSTO
CUSTOMER_NAME				

0104	04-SEP-26	3200	Pending	C004
Vijay				
0105	05-SEP-26	2750	Shipped	C005
Abi				
0106	06-SEP-26	1200	Delivered	C006
Anu				

2. Find orders where customer details are missing.

```
SQL> SELECT O.ORDER_ID AS ORDER_ID, O.ORDER_DATE AS ORDER_DATE, O.TOTAL_AMOUNT AS TOTAL_AMOUNT, O.ORDER_STATUS AS ORDER_STATUS FROM CUSTOMER C RIGHT JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID WHERE C.CUSTOMER_ID IS NULL;
```

no rows selected

3. Display all payment records with order details.

```
SQL> SELECT P.PAYMENT_ID, P.PAYMENT_DATE, P.PAYMENT_MODE, P.PAYMENT_STATUS, P.TRANSACTION_AMOUNT, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT, O.ORDER_STATUS FROM ORDERS O RIGHT JOIN PAYMENT P ON O.ORDER_ID = P.ORDER_ID ORDER BY P.PAYMENT_DATE;
```

PAYMENT_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS	
TRANSACTION_AMOUNT	ORDER_I	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS

1	20-SEP-26	UPI	Success	
2500	0101	01-SEP-26	1500	Pending
2	21-SEP-26	Card	Success	
1800	0102	02-SEP-26	2500	Shipped
3	22-SEP-26	Cash	Pending	
3200	0103	03-SEP-26	1800	Delivered

PAYMENT_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS	
TRANSACTION_AMOUNT	ORDER_I	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS

4	23-SEP-26	UPI	Success	
1500	0104	04-SEP-26	3200	Pending
5	24-SEP-26	Card	Success	
2750	0105	05-SEP-26	2750	Shipped

5. Retrieve Complete Order Details Task:

Generate a complete order report by combining:

Customer Name

Product Name

Quantity

Order Date

Total Amount

Payment Status

```
SQL> SELECT C.CUSTOMER_NAME, P.PRODUCT_NAME, OD.QUANTITY, O.ORDER_DATE, O.TOTAL_AMOUNT, PM.PAYMENT_STATUS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID JOIN ORDER_DETAILS OD ON O.ORDER_ID = OD.ORDER_ID JOIN PRODUCT P ON OD.PRODUCT_ID = P.PRODUCT_ID LEFT JOIN PAYMENT PM ON O.ORDER_ID = PM.ORDER_ID ORDER BY O.ORDER_DATE, C.CUSTOMER_ID;
```

CUSTOMER_NAME	PRODUCT_NAME	QUANTITY	ORDER_DAT	TOTAL_AMOUNT	PAYMENT_STATUS
Devi	Paracetamol	2	01-SEP-26	1500	Success
Radhi	Amoxicillin	3	02-SEP-26	2500	Success
Ajay	Cetirizine	2	03-SEP-26	1800	Pending
Vijay	Omeprazole				

6.Display Customer Purchase History

1. All products purchased by a customer.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, ROUND(SUM(O.TOTAL_AMOUNT), 2) AS TOTAL_AMOUNT_SPENT FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME HAVING SUM(O.TOTAL_AMOUNT) = (SELECT MAX(TOTAL_SPENT) FROM (SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENT FROM ORDERS GROUP BY CUSTOMER_ID));
```

CUSTO	CUSTOMER_NAME	TOTAL_AMOUNT_SPENT
C001	Devi	5000

2. Total amount spent by each customer.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS TOTAL_ORDERS, ROUND(SUM(O.TOTAL_AMOUNT), 2) AS TOTAL_AMOUNT_SPENT FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY TOTAL_AMOUNT_SPENT DESC;
```

CUSTO	CUSTOMER_NAME	TOTAL_ORDERS	TOTAL_AMOUNT_SPENT
C001	Devi	2	5000
C007	Sindhu	1	4500
C002	Radhi	2	4400
C004	Vijay	1	3200
C005	Abi	1	2750
C009	Sanjay	1	2100
C003	Ajay	1	1800
C006	Anu	1	1200

8 rows selected.

3. Number of orders placed by each customer

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS TOTAL_ORDERS FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY TOTAL_ORDERS DESC;
```

CUSTO	CUSTOMER_NAME	TOTAL_ORDERS
C001	Devi	2
C002	Radhi	2
C004	Vijay	1
C007	Sindhu	1
C003	Ajay	1
C009	Sanjay	1
C006	Anu	1
C005	Abi	1

8 rows selected.

4. Latest purchase details of customers.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT, O.ORDER_STATUS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID WHERE O.ORDER_DATE = (SELECT MAX(O2.ORDER_DATE) FROM ORDERS O2 WHERE O2.CUSTOMER_ID = C.CUSTOMER_ID);
```

CUSTO	CUSTOMER_NAME	ORDER_I	ORDER_DAT	TOTAL_AMOUNT
C001	Devi	0109	09-SEP-26	3500
C002	Radhi	0110	10-SEP-26	1900
C003	Ajay	0103	03-SEP-26	1800
C004	Vijay	0104	04-SEP-26	3200
C005	Abi	0105	05-SEP-26	2750
C006	Anu	0106	06-SEP-26	1200
C007	Sindhu	0107	07-SEP-26	4500

7. Generate Multi-Table Business Report

Report 1: Customer Order Report

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE, O.ORDER_STATUS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID ORDER BY O.ORDER_DATE;
```

CUSTO	CUSTOMER_NAME	ORDER_I	ORDER_DAT	ORDER_STATUS
C001	Devi	0101	01-SEP-26	Pending
C002	Radhi	0102	02-SEP-26	Shipped
C003	Ajay	0103	03-SEP-26	Delivered
C004	Vijay	0104	04-SEP-26	Pending
C005	Abi	0105	05-SEP-26	Shipped
C006	Anu	0106	06-SEP-26	Delivered
C007	Sindhu	0107	07-SEP-26	Pending
C009	Sanjay	0108	08-SEP-26	Shipped
C001	Devi	0109	09-SEP-26	Delivered
C002	Radhi	0110	10-SEP-26	Pending

Report 2: Sales Report

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, SUM(OD.QUANTITY) AS QUANTITY_SOLD, ROUND(SUM(OD.QUANTITY * OD.PRICE), 2) AS TOTAL_REVENUE FROM PRODUCT P JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME ORDER BY TOTAL_REVENUE DESC;
```

PRODUCT	PRODUCT_NAME	QUANTITY_SOLD	TOTAL_REVENUE
P107	Antiseptic Cream	3	4500
P109	Metformin	5	3500
P104	Omeprazole	4	3200
P105	Vitamin C		

Report 3: Payment Analysis Report

```
SQL> SELECT PAYMENT_MODE, COUNT(PAYMENT_ID) AS NUMBER_OF_TRANSACTIONS, SUM(CASE WHEN PAYMENT_STATUS = 'SUCCESSFUL' THEN 1 ELSE 0 END) AS SUCCESSFUL_PAYMENTS FROM PAYMENT GROUP BY PAYMENT_MODE;
```

PAYMENT_MODE	NUMBER_OF_TRANSACTIONS	SUCCESSFUL_PAYMENTS
Card	2	0
Cash	1	0
UPI	2	0

Report 4: Customer Purchase Analysis

(top purchasing order)

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY TOTAL_SPENDING DESC;
```

CUSTO	CUSTOMER_NAME	TOTAL_SPENDING
C001	Devi	5000
C007	Sindhu	4500
C002	Radhi	4400
C004	Vijay	3200
C005	Abi	2750
C009	Sanjay	2100
C003	Ajay	1800
C006	Anu	1200

Maximum orders

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS TOTAL_ORDERS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME HAVING COUNT(O.ORDER_ID) = (SELECT MAX(ORDER_COUNT) FROM (SELECT COUNT(ORDER_ID) AS ORDER_COUNT FROM ORDERS GROUP BY CUSTOMER_ID));
```

CUSTO	CUSTOMER_NAME	TOTAL_ORDERS
C001	Devi	2
C002	Radhi	2

Customers with highest spending

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, ROUND(SUM(O.TOTAL_AMOUNT), 2) AS TOTAL_AMOUNT_SPENT FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME HAVING SUM(O.TOTAL_AMOUNT) = (SELECT MAX(TOTAL_SPENT) FROM (SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENT FROM ORDERS GROUP BY CUSTOMER_ID));
```

CUSTO	CUSTOMER_NAME	TOTAL_AMOUNT_SPENT
C001	Devi	5000